Evaluating Temporal Dynamics and Attention Mechanisms in GNN Session-based Recommendation

Angel E. Rodríguez-Román, Rolando Menchaca-Méndez, Ricardo Menchaca-Méndez, and Luis P. Sánchez-Fernández

Centro de Investigación en Computación del Instituto Politécnico Nacional, Ciudad de México 07700, México
{arodriguezr2024, rmen, ric, lsanchez}@cic.ipn.mx

Abstract. Session-based recommendation systems increasingly rely on Graph Neural Networks (GNNs) to capture complex item transitions. While some approaches suggest incorporating time dynamics to improve recommendations, the actual impact of these temporal features remains under-evaluated. In this work, we systematically investigate integrating temporal information into GNN-based recommenders via a gated mechanism. Our findings reveal that explicit temporal data often degrades overall model performance, acting as noise during optimization. To address this, we introduce a learnable parameter that dynamically weights the importance of temporal versus structural information within sessions. Our evaluation confirms that optimal performance is frequently achieved by actively down-weighting temporal features. Beyond our temporal analysis, we adapt a modified Gated Recurrent Unit (GRU) that demonstrates improved performance over standard GNN baselines on specific datasets. Finally, we explore the use of dot-product attention in place of the widely adopted additive attention, achieving competitive results. Ultimately, this work provides insights into the limitations of time dynamics and offers architectural improvements for session-based GNNs.

Keywords: Computational Intelligence · Recommender Systems · Session-based Recommendation · Deep Learning · Graph Deep Learning

1 Introduction

Recommendation Systems (RS) have become an important part of modern digital platforms, serving as a mechanism to alleviate information overload. By filtering vast amounts of available data, these systems enable users to navigate complex catalogs and discover relevant content efficiently. Consequently, RS are now indispensable across a wide spectrum of industry applications, including e-commerce platforms, multimedia streaming services, and social networks [1], [2].

Within this area of research, Session-Based Recommendation (SBR) has emerged as a critical specialized task [6][10]. SBR focuses on predicting the

next item a user will interact with, based on a short, ongoing sequence of actions. For example, in an e-commerce setting, an SBR model analyzes a user's current sequence of product clicks to accurately forecast the next item they are most likely to view or purchase. This approach is indispensable in anonymous contexts, where profile information is unavailable, and one must rely solely on session information.

To effectively model the complex, non-linear item transitions within these short sequences, some similar approaches have successfully applied Graph Neural Networks (GNNs) to SBR tasks. By representing user sessions as directed graphs, GNNs can capture structural dependencies between items. Building on this structural foundation, several architectures have sought to further integrate temporal dynamics into the graph, assuming that interaction timing provides critical predictive context [12] [13] [14].

However, the precise impact of this temporal information remains underevaluated. In this paper, we question the prevailing assumption that time dynamics inherently improve GNN-based recommenders. Our analysis reveals that explicitly incorporating time often introduces noise during optimization, degrading model performance. To address this, we propose a more detailed evaluation of the impact of incorporating temporal features and introduce a set of architectural refinements to current GNN-based SBR models.

The rest of the paper is organized as follows. First, section 2 explores existing literature. From classical approaches to contemporary deep learning methods. Section 3 then formalizes the problem statement and details the proposed end-to-end architecture, providing a comprehensive breakdown of its underlying components. Subsequently, section 4 describes the experimental framework, including dataset characteristics and an analysis of the obtained results. Finally, section 5 presents conclusions and potential directions for future research.

2 Related work

2.1 Traditional Recommendation Techniques

Early work has focused on traditional algorithms that are used in other domains. K-Nearest-Neighbor-based (KNN-based) recommendation systems have been widely used in traditional recommendation systems. The goal is to rank items to provide recommendations. Unfortunately, these models do not consider either the order of events or the sequential nature of sessions. Despite this, KNN-based recommenders have served as a strong baseline in session-based recommendation. In [8], the authors proposed STAN (Sequence and Time Aware Neighborhood). It takes into account three factors: the position of an item in the current session, the recency of a past session with respect to the current session, and the position of a recommendable item in a neighboring session. The importance of the mentioned factors is adjusted via controllable decay factors. The results obtained are comparable to those reported in similar deep learning approaches.

2.2 Markov-Decision-Process-based

Other works have modeled the problem as a Markov Decision Process (MDP). Such as [7], which proposed an MDP-based recommendation system. They argue that the recommendation problem can be modeled as a sequential process. By adopting an MDP-based approach, the problem becomes similar to the reinforcement learning framework. They expect systems to maximize the long-term utility and expected value of a recommendation, rather than merely the best immediate clicks. They utilized a sophisticated n-gram predictive model that functions as a Markov chain to initialize the system. This model incorporates enhancements such as skipping, similarity clustering, and finite mixture modeling to effectively handle sparse data. Challenges with MC in session-based recommendation include failing to account for historical events, assuming independence, and the growing cardinality of the state space when the model considers all possible paths/sequences.

2.3 Deep Learning Approaches

Due to their ability to model sequential data, the authors in [6] were among the first to tackle session-based recommendation using recurrent neural networks (RNNs). They propose an end-to-end architecture named GRU4REC that uses a Gated Recurrent Unit (GRU) and adapts two ranking loss functions: The Bayesian Personalized Ranking (BPR), which is a matrix factorization method that uses pairwise ranking loss, and TOP1, which is the regularized approximation of the relative rank of the relevant item.

Another important work based on RNNs is [5]. Here, the authors address the limitations of traditional deep learning models by proposing a temporal gating methodology that integrates the time intervals between user interactions. They proposed a Time-aware GRU (a generalization of a traditional GRU) to handle temporal information, and a time-aware attention mechanism to query long-term historical records in a weighted manner, using time as the weight during attention score computation.

The approach using the graph neural network (GNN) framework typically involves building a directed graph for each session and then applying GNNs to learn item embedding representations. The main difference between GNNs and traditional deep learning architectures like [A MLP session-based recommendation] is that GNNs capture the graph's structure via neighborhood aggregation, in sequential recommendation, sessions are modeled as graphs, and GNNs have been shown to capture richer embedding representations. [3] was the first work that used GNNs for sequential recommendation. The authors proposed a Gated GNN (GRU-based) to model sequential interactions. The proposed architecture outperformed traditional methods. In 2022, based on [3], Tajuddeen et al. Proposed a similar architecture in [4]. The main difference in the architecture is that it takes non-sequential information into account and uses a traditional GNN to first generate embeddings for non-sequential information. Then, the embeddings are fed into the Gated GNN as inputs.

3 Solution Approach

3.1 Problem statement

We define the problem as predicting the next interaction in an anonymous sequence. Let V denote the set of items and S the set of sessions. Formally, each session $s \in S$ is represented as $s = [(v_{s1}, t_{s1}), (v_{s2}, t_{s2}), \ldots, (v_{sn-1}, t_{sn-1})]$, where $v_{si} \in V$ is the visited item and t_{si} its corresponding timestamp. The objective is to leverage both the item transitions and their temporal intervals to predict the most likely succeeding item $v_{sn} \in V$. Figure 3.1 illustrates a high-level view of the proposed architecture.

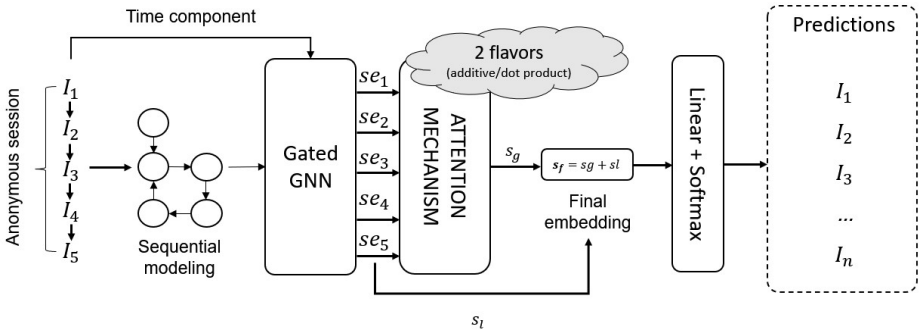

Fig. 1. High-level diagram of the architecture

3.2 Session graph construction

For each session $s \in S$, we construct a directed session graph represented by two distinct structures: $G_{s_{in}}$ and $G_{s_{out}}$. In both cases, each node represents an item $v_{si} \in V_s$ within the session. The first graph, $G_{s_{in}} = (V_s, E_{in})$, defines the edges as $E_{in} = \{(v_{s,i-1}, v_{si})\}$, representing the incoming transitions in the sequence. Similarly, $G_{s_{out}} = (V_s, E_{out})$ is constructed with edges $E_{out} = \{(v_{si}, v_{s,i+1})\}$, representing the outgoing links. These two representations, formalized as adjacency matrices A_{in} and A_{out}, are concatenated to form a unified input for the GNN. This dual-graph approach enables the model to simultaneously propagate information from both preceding and succeeding items, enriching the latent representation of each node.

3.3 Time aware modeling

To incorporate temporal dynamics, we apply a preprocessing workflow to the session timestamps. Given a session $s = [(v_{s1}, t_{s1}), (v_{s2}, t_{s2}), \ldots, (v_{sn}, t_{sn})] \in S$, we derive a sequence of time intervals $\tau = \{\Delta t_{si}\}_{i=2}^{n}$, where each element represents the duration between adjacent interactions, defined as $\Delta t_{si} = t_{si} -$

t_{si-1}. To account for the long-tailed distribution of these intervals, a logarithmic transformation is applied: $t'_{si} = \log(\Delta t_{si} + 1)$. Subsequently, these values are mapped to the range $[0, 1]$ via min-max normalization. The resulting temporal features are then utilized to compute a time gate g_s:

$$\delta_s = \phi(\tau \odot W_{\delta_s} + b_{\delta_s}) \tag{1}$$

$$\lambda_s = \phi([x_s, h_{s-1}]W_\lambda + b_\lambda) \tag{2}$$

$$\hat{g_s} = \sigma(\delta_s \odot W_{g_s\delta_s} + \lambda_s \odot W_{g_s\lambda_s} + b_{g_s}) \tag{3}$$

In this formulation, δ_s is designed to capture the non-linear interactions within the temporal information τ_s, while λ_s aims to extract the structural semantic information from both the aggregated neighborhood message and the previous hidden state. A learnable parameter α is proposed to weigh the importance of temporal information versus semantic information. α is initialized with the value 0. And after applying a sigmoid function. We set it to 0.5, meaning that the same importance is given to both signals (temporal and structural). Then $\hat{g_s}$ is used to compute the final representation yielding a protected time gate:

$$g_s = \alpha * \hat{g_s} + (1 - \alpha) \tag{4}$$

3.4 Sequential modeling

The Gated Graph Neural Network (GGNN) integrates the structural information of the session graph with the item-level latent representations. Unlike standard Graph Convolutional Networks (GCNs), the GGNN incorporates a recurrent gating mechanism to effectively manage sequential dependencies and to collectively update node embeddings. First, the message passing step aggregates information from the neighborhood for all nodes in the session simultaneously. The incoming and outgoing representations are linearly transformed and concatenated to form the aggregated message matrix a_s:

$$a_s = [A_{in}h_{s-1}W_{in}, A_{out}h_{s-1}W_{out}] \tag{5}$$

where A_{in} and A_{out} denote the incoming and outgoing adjacency matrices of the session graph, h_{s-1} is the matrix of node hidden states from the previous iteration, and $[\cdot, \cdot]$ represents the concatenation operation. Subsequently, the update gate z_s, reset gate r_s, the candidate state $\tilde{h}_s$ and the final state h_s are computed by concatenating the message matrix a_s with the previous hidden state h_{s-1} as follows:

$$z_s = \sigma([a_s, h_{s-1}]W_z + b_z) \tag{6}$$

$$r_s = \sigma([a_s, h_{s-1}]W_r + b_r) \tag{7}$$

$$\tilde{h}_s = \tanh([a_s, (r_s \odot h_{s-1})]W_h + b_h) \tag{8}$$

$$h_s = z_s \odot h_{s-1} + (1 - z_s) \odot \tilde{h}_s \tag{9}$$

Finally, the hidden state is updated by modulating the previous state with both the standard update gate z_s and the protected time gate g_s from equation 4, combining the historical temporal context with the new candidate state, therefore, equation 9 is rewritten as:

$$h_s = z_s \odot (g_s \odot h_{s-1}) + (1 - z_s) \odot \tilde{h}_s \tag{10}$$

3.5 Attention Mechanism

Additive Attention. To establish a baseline for performance comparison, we formalize the additive attention mechanism as utilized in [3] and [4]. In this approach, the global preference vector s_g is computed as a weighted sum of the node hidden states:

$$s_g = \sum_{i=1}^{n} \alpha_i h_i \tag{11}$$

where n is the sequence length, and each α_i represents the attention weight for the item embedding at position i. These weights are calculated using a single-layer feed-forward network: $\alpha_i = q^T \sigma(W_1 h_n + W_2 h_i + b)$. Here, h_n represents the embedding of the last interacted item (acting as the local preference), while q, W_1, and W_2 are learnable parameters that control the additive attention distribution.

Dot Product Attention. This work adopts the scaled dot-product attention mechanism [9] both to evaluate its performance against the additive baseline and to provide a flexible framework for future feature integration. To compute the global session preference s_g, we map the GGNN output representations—specifically, the session matrix $H \in \mathbb{R}^{n \times d}$ and the last interacted item $h_n \in \mathbb{R}^d$—into Query (Q), Key (K), and Value (V) matrices via learnable weights $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$:

$$Q = h_n W_Q, \quad K = H W_K, \quad V = H W_V \tag{12}$$

In this formulation, Q represents the user's recent intent, while K and V capture the semantic features of the historical session context. The global preference s_g is then computed as a weighted sum of the values (V), where the attention scores are derived from the scaled inner product between Q and K:

$$s_g = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right) V \tag{13}$$

3.6 Prediction

To predict the next interaction, the final hybrid representation of the session, s_f, is computed by summing (element-wise) the local preference s_l (the last item embedding) and the global preference s_g derived from the attention mechanism:

$$s_f = s_l + s_g \tag{14}$$

Subsequently, the recommendation score $\hat{z}_i$ for each candidate item $v_i \in V$ is calculated via its inner product with the final session representation:

$$\hat{z}_i = s_f^T v_i \tag{15}$$

These raw scores are then normalized into a probability distribution $\hat{\mathbf{y}} = softmax(\hat{z})$. For any given session, the categorical cross-entropy loss function between the predictions and the ground truth is used:

$$\mathcal{L}(\hat{\mathbf{y}}) = -\sum_{i=1}^{|V|} y_i \log(\hat{y}_i) \tag{16}$$

Where $\mathbf{y}$ denotes the ground truth one-hot vector, $\hat{\mathbf{y}}$ is the vector of predicted probabilities for all V items and $\hat{y}_i$ is the probability assigned to item i. Since the ground truth for each session is a one-hot encoded vector all elements y_i are zero except for the one corresponding to the actual next item in the session ($y_j = 1$). Consequently, Equation 16 simplifies to the negative log-likelihood of the true item:

$$\mathcal{L}(\hat{\mathbf{y}}) = -\log(\hat{y}_j) \tag{17}$$

where $\hat{y}_j$ represents the predicted probability for the ground truth item j.

4 Experimental Results

4.1 Dataset

We evaluated our model on two real-world datasets containing e-commerce transactions: **Diginetica** and **Yoochoose 1/64** (a standard fraction of the massive Yoochoose dataset). Diginetica is a retail technology company. The Diginetica dataset is from the CIKM 2016 Cup. It has 573,935 sessions, 134,319,529 products, 18,025 purchases, and contains TimeStamp information. The Yoochoose dataset contains information on six months of activity from a large e-commerce business in Europe selling a wide range of products, including garden tools, toys, clothes, electronics, and more. The dataset is from the RecSys Challenge 2015, a collection of sequences of click events, click sessions, and purchase events. It contains 9,249,729 sessions, 52,739 products, 1,150,753 purchases, and TimeStamp information. For a fair comparison with [3], the last 7 days of information are taken for testing, also sessions of length 1 and items appearing fewer than 5 times are filtered out. Data augmentation is performed to generate sets of subsessions. Specifically, for a session $s = [v_{s1}, v_{s2}, \ldots, v_{sn}] \in S$, we created multiple sequence-target pairs defined as $\{([v_{s1}, \ldots, v_{si}], v_{s,i+1}) \mid 1 \leq i < n\}$.After preprocessing, we obtained 924,259 sessions for Diginetica and 425,757 sessions for Yoochoose 1/64. Finally, 10% of the training data was randomly sampled to create the validation set. The final statistics for both datasets are summarized in Table 4.1:

Table 1. Statistics of Yoochoose and Diginetica datasets after preprocessing

Dataset	Total sessions	Training	Testing	Items	Avg. Session len
Diginetica	924,259	719,470	204,789	43097	4.85
Yoochoose 1/64	425,757	369,859	55,898	37484	3.9727

4.2 Experimental setup

For all the experiments, a hidden size $d = 100$ (representing the dimension of item embeddings in the latent space) and a batch size of 100 (e.g., 100 sessions per batch) were used. An initial learning rate (lr) of 0.001 with learning rate decay of 0.1 every 3 steps was used. l2 penalty $l2 = 1 \times 10^{-5}$ was used as a regularization term to prevent overfitting and potentially improve generalization. The number of epochs was set to 30 and a patience parameter was used as an early stopping criterion.
For evaluation, 2 metrics were considered *Precision@20* (also known as *Hit Rate*) and Mean Reciprocal Rank *MRR*@20. All the parameters of the model were normalized with a mean of 0 and a standard deviation of 0.1. $Precision$@20 captures the number of times a target element is within the provided recommendations by the model

$$Precision@20 = \frac{\lambda_{hit}}{N} \tag{18}$$

Where λ_{hit} is the number of times the prediction was within the top 20 recommendations.

MRR@20 evaluates the quality of the recommendation by taking into account the position of the recommendation.

$$MRR@K = \frac{1}{N} \sum_{i \in Rec}^{N} \frac{1}{rank(i)} \tag{19}$$

Where i is an item in the ranking of the recommendation list. And $rank(i)$ tells us the position of the item in the list.

4.3 Results

We evaluated the proposed architecture on both datasets to assess the adapted GRU mechanism [5] and the overall impact of the temporal component. As defined in Equation 4, the learnable parameter α dynamically balances the influence of this temporal data against the structural graph information during optimization. To ensure robustness and reproducibility, we conducted 8 distinct experiments, each repeated 5 times with random seeds 40-44. The mean and standard deviation for these runs are summarized in Table 2. Additionally, table 3 and figure 2 show the behavior of α during the optimization process across the experiments.

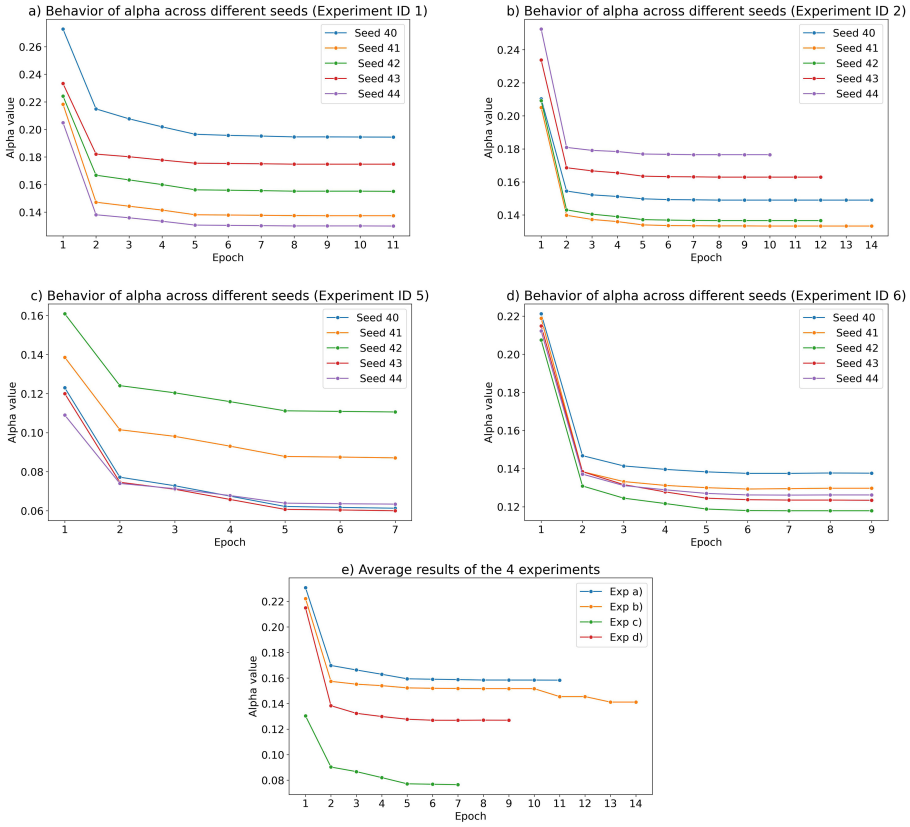

Fig. 2. Illustration of the behavior of α during the number of epochs across the different time-aware experiments during the training process. As illustrated in Figures (a) and (b), which correspond to the additive and scaled dot-product attention mechanisms for the Yoochoose dataset, the parameter α exhibits a consistent downward trajectory across both experiments. Despite the stochastic nature of parameter initialization, α monotonically decreases as training progresses, ultimately converging to values below 0.14. Similarly, for the Diginetica dataset, Figures (c) and (d) show a significant decrease, converging to values of α below 0.07 and 0.12, respectively. Finally, Figure (e) summarizes all the charts and shows the average values of α, confirming the same pattern. The fact that this behavior remains invariant to the choice of attention mechanism suggests that the suppression of the temporal signal is a fundamental response of the time-aware gated graph neural network implemented in this work.

Table 2. Results of all experiments on the Diginetica and Yoochoose 1/64 datasets.

ID	Dataset	Time Aware	Attention	*P*@20	*MRR*@20
1	Yoochoose	Yes	Additive	70.03 ± 0.12	30.34 ± 0.06
2	Yoochoose	Yes	Dot Product	70.32 ± 0.07	30.47 ± 0.07
3	Yoochoose	No	Additive	70.13 ± 0.08	30.63 ± 0.07
4	Yoochoose	No	Dot product	70.32 ± 0.06	30.49 ± 0.05
5	Diginetica	Yes	Additive	51.00 ± 0.04	17.71 ± 0.06
6	Diginetica	Yes	Dot Product	50.55 ± 0.10	17.72 ± 0.05
7	Diginetica	No	Additive	51.13 ± 0.09	17.74 ± 0.06
8	Diginetica	No	Dot Product	50.67 ± 0.06	17.70 ± 0.04

Table 3. Behavior of the parameter alpha during the optimization process. α is initialized to 0.5 for all experiments; the average value after the 1st epoch is shown for each experiment, as well as the average last value.

Exp. ID	Init α	Value after 1st epoch	Last value
1	0.5	0.2307	0.1583
2	0.5	0.2221	0.1516
5	0.5	0.1303	0.0764
6	0.5	0.2148	0.1269

4.4 Ablation study

The experimental results, summarized in the preceding tables, evaluate the impact of the temporal component in the proposed architecture and compare the effectiveness of the attention mechanisms.

Impact of temporal information. In the context of the Diginetica and Yoochoose datasets, integrating temporal information does not yield performance improvements. Instead, the temporal intervals appear to introduce noise during the optimization process, degrading both Precision and MRR. This empirical observation is corroborated by the behavior of the learnable parameter α (Table 3 and figure 2). Although initialized at 0.5, α experiences a decline after the first training epoch and consistently converges to marginal values (ranging from 0.076 to 0.158) by the end of training. This marked decay indicates that the network actively learns to suppress the temporal component, prioritizing structural graph information. Consequently, optimal results are achieved for both datasets when the time-aware component is completely omitted (Experiments 3, 4, 7, and 8).

Influence of the Attention Mechanism. As described earlier, we also evaluated 2 different attention mechanisms to determine the effect of this component on the architecture. The results (Table 2) showed similar performance across the different trials. In experiments 1 to 4, the Dot-Product approach slightly outperforms the Additive mechanism in Precision@20 (70.32 vs. 70.13), while

yielding a lower MRR@20 (30.63 vs 30.49). Conversely, in experiments 5 to 8, the additive attention maintains an advantage across both metrics (but particularly in P@20). Although we tested on only 2 datasets, these results suggest that the choice of attention mechanism for better performance is dataset-dependent and that Scaled Dot-Product attention is a highly viable alternative to the additive baseline, with potential for further improvements in this area. Since this mechanism has been widely used and adapted for current work in session-based recommendation [18–22] and other fields, such as large language models for next-token prediction [15–17].

4.5 Comparison with baseline methods

To compare our results with baseline algorithms, we select experiments 4 and 7 for the Diginetica and Yoochoose datasets, respectively. We considered the following baseline algorithms for comparison:

- **POP:** This method recommends the most popular item in the dataset.
- **S-POP:** Similar to POP, this method is a session-based algorithm that recommends the most popular item in the session.
- **item-KNN:** This method uses metrics (e.g., cosine similarity) to find and recommend items similar to the last in the session.
- **BPR-MF:** It is a matrix factorization (MF) method. Although MF is not used for session-based recommendation, item embeddings within a session can be learned via a pairwise ranking loss, thereby optimizing the latent representation of items for recommendation.
- **FPMC:** It is an MC-based method for sequential recommendation and next-basket-prediction.
- **GRU4REC:** It is a RNN-based method. It uses a GRU model and a pairwise ranking loss function to perform recommendations.
- **NARM:** It is a transformer-based model. It uses an encoder-decoder architecture for sequence-aware recommendation [10]
- **STAMP:** This uses an attention-based model. It tries to capture both. The short and long-term preferences in a sequence for recommendation.

Table 4 shows the results comparing experiments 4 and 7 with the baseline methods mentioned above (Best results in bold).

Table 4. Comparison of the proposed method against baseline models. Best results are highlighted in bold.

	Yoochoose		**Diginetica**	
Method	P@20	MRR@20	P@20	MRR@20
POP	6.71	1.65	0.89	0.20
S-POP	30.44	18.35	21.06	13.68
Item-KNN	51.60	21.81	35.75	11.57
BPR-MF	31.31	12.08	5.25	1.98
FPMC	45.62	15.01	26.53	6.95
GRU4REC	60.64	22.89	29.45	8.33
NARM	68.32	28.63	49.70	16.17
STAMP	68.74	29.67	45.64	14.32
SR-GNN (Reported)	**70.57**	**30.94**	50.73	17.59
SR-GNN (Tested)	70.11	30.64	50.20	17.43
Ours	70.32	30.49	**51.13**	**17.74**

The results indicate that our proposed method outperforms almost all baselines, including SR-GNN, on the Diginetica dataset across both performance metrics. Notably, when compared against the SR-GNN results obtained under our specific experimental conditions (SR-GNN Tested), our method achieves superior performance on both datasets. We hypothesize that our adapted GRU architecture facilitates more effective projection into the latent space, yielding richer embeddings than the standard GRU used in SR-GNN. This suggests that existing frameworks, such as [3], [4], [12], and [13], could potentially improve their results by integrating the GRU mechanism proposed in this study.

5 Conclusions and future work

This study demonstrates that our proposed architecture provides a robust session-based recommendation mechanism compared to standard implementations. By refining hidden-state transitions to better capture sequential dependencies, the model consistently outperforms the SR-GNN baseline and other established methods across both datasets under identical testing conditions. Furthermore, our investigation into temporal dynamics via the learnable α parameter revealed that, for the Yoochoose and Diginetica datasets, temporal information did not significantly improve predictive performance. The notable decay of α toward zero indicates that the network actively prioritizes structural and sequential signals over temporal data, which primarily acted as optimization noise. These results validate our architectural improvements to the GRU and provide empirical evidence of the limitations of time-aware features in these specific recommendation contexts.

We propose the following directions for future research: Generate synthetic data specifically designed to highlight time-series dynamics, to assess empirically whether the proposed architecture effectively leverages temporal information. Evaluate the proposed framework across additional domains, such as music,

movies, and news, to provide broader empirical evidence on whether temporal signals consistently act as noise or are domain-dependent. Provide a rigorous characterization of the convergence behavior of the gating parameter α. Specifically, investigating why α tends toward zero to help clarify the conditions under which the architecture prioritizes structural information over temporal signals. Time-Aware Attention: Explore time dynamics within the attention mechanism. Similar to the gating logic used in this work, we aim to compute a temporal gate during the attention phase to make the mechanism more sensitive to time intervals. Modeling Non-Sequential Information: Take advantage of Graph Neural Networks (GNNs) to better capture the graph structure of sessions. This could lead to strategically combining sequential and non-sequential information to obtain richer embedding representations and further improve accuracy. Hybrid Architectures: Implement a novel architecture that combines the strengths of attention mechanisms with GNNs, as this combination has proven robust for session-based recommendation tasks.

References

1. Aggarwal, C.C.: Recommender Systems: The Textbook. 1st edn. Springer, Cham (2016)
2. Nesmaoui, R., Louhichi, M., Lazaar, M.: A Collaborative Filtering Movies Recommendation System based on Graph Neural Network. Procedia Computer Science **220**, 456–461 (2023). https://doi.org/10.1016/j.procs.2023.03.058
3. Wu, S., Tang, Y., Zhu, Y., Wang, L., Xie, X., Tan, T.: Session-Based Recommendation with Graph Neural Networks. In: Proceedings of the Thirty-Third AAAI Conference on Artificial Intelligence (AAAI-19), pp. 346–353. AAAI Press (2019)
4. Gwadabe, T.R., Liu, Y.: Improving graph neural network for session-based recommendation system via non-sequential interactions. Neurocomputing **468**, 143–155 (2022)
5. Ji, W., Wang, K., Wang, X., Chen, T., Cristea, A.: Sequential Recommender via Time-aware Attentive Memory Network. In: Proceedings of the 29th ACM International Conference on Information and Knowledge Management (CIKM '20), pp. 565–574. ACM, Virtual Event (2020). https://doi.org/10.1145/3340531.3411869
6. Hidasi, B., Karatzoglou, A., Baltrunas, L., Tikk, D.: Session-based Recommendations with Recurrent Neural Networks. In: International Conference on Learning Representations (ICLR) (2016)
7. Shani, G., Brafman, R.I., Heckerman, D.: An MDP based Recommender System. Journal of Machine Learning Research **6**, 1265–1305 (2005)
8. Garg, D., Gupta, P., Malhotra, P., Vig, L., Shroff, G.: Sequence and Time Aware Neighborhood for Session-based Recommendations: STAN. In: Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1069–1072. ACM, New York (2019). https://doi.org/10.1145/3331184.3331322
9. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L., Polosukhin, I.: Attention Is All You Need. In: Advances in Neural Information Processing Systems (NeurIPS 2017), pp. 5998–6008 (2017)
10. Li, J., Ren, P., Chen, Z., Ren, Z., Lian, T., Ma, J.: Neural Attentive Session-based Recommendation. In: Proceedings of the 2017 ACM International Conference on

Information and Knowledge Management (CIKM '17), pp. 1419–1428. ACM, New York (2017). https://doi.org/10.1145/3132847.3132926
11. Tan, Y.K., Xu, X., Liu, Y.: Improved Recurrent Neural Networks for Session-based Recommendations. In: Proceedings of the 1st Workshop on Deep Learning for Recommender Systems (DLRS), pp. 17–22. ACM (2016)
12. Huang, B., Bi, Y., Wu, Z., Wang, J., Xiao, J.: UBER-GNN: A User-Based Embeddings Recommendation based on Graph Neural Networks. Preprint at https://arxiv.org/abs/2008.02546 (2020)
13. Yu, F., Zhu, Y., Liu, Q., Wu, S., Wang, L., Tan, T.: TAGNN: Target Attentive Graph Neural Networks for Session-based Recommendation. In: Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval, pp. 1921–1924. ACM (2020). https://doi.org/10.1145/3397271.3401319
14. Song, W., Xiao, Z., Wang, Y., Charlin, L., Zhang, M., Tang, J.: Session-based Social Recommendation via Dynamic Graph Attention Networks. In: Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining (WSDM '19), pp. 555–563. ACM (2019). https://doi.org/10.1145/3289600.3290989
15. Sanchis-Agudo, M., Wang, Y., Arnau, R., Guastoni, L., Lim, J., Duraisamy, K., Vinuesa, R.: Easy attention: A simple attention mechanism for temporal predictions with transformers. Preprint at https://arxiv.org/abs/2308.12874 (2025)
16. Li, Y., Huang, Y., Ildiz, M.E., Rawat, A.S., Oymak, S.: Mechanics of Next Token Prediction with Self-Attention. Preprint at https://arxiv.org/abs/2403.08081 (2024)
17. Brauwers, G., Frasincar, F.: A General Survey on Attention Mechanisms in Deep Learning. IEEE Transactions on Knowledge and Data Engineering **35**(4), 3279–3298 (2023). https://doi.org/10.1109/tkde.2021.3126456
18. Li, [Iniciales por confirmar]: Collaborative local–global context modeling for session-based recommendation. Information Processing & Management **62**(5), 104196 (2025). https://doi.org/10.1016/j.ipm.2025.104196
19. Zhang, X., He, B., Chen, J., Cui, Z., Ma, C.: From Token to Item: Enhancing Large Language Models for Recommendation via Item-aware Attention Mechanism. Preprint at https://arxiv.org/abs/2603.19693 (2026)
20. Liu, Z., Liu, S., Zhang, Z., Cai, Q., Zhao, X., Zhao, K., Hu, L., Jiang, P., Gai, K.: Sequential Recommendation for Optimizing Both Immediate Feedback and Long-term Retention. In: Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24), pp. 1872–1882. ACM (2024). https://doi.org/10.1145/3626772.3657829
21. Bai, X., Peng, H., Huang, Y., Wang, J., Yang, Q., Ramírez-De-Arellano, A.: A graph attention network integrated with gated spiking neural P systems for session-based recommendation. Expert Systems with Applications **286**, 128029 (2025). https://doi.org/10.1016/j.eswa.2025.128029
22. Yang, F., Peng, D.: A graph neural network with topic relation heterogeneous multi-level cross-item information for session-based recommendation. Information Systems **123**, 102380 (2024). https://doi.org/10.1016/j.is.2024.102380